GUIA DE REFERÊNCIA

Git & GitHub



Terminal · Git · GitHub — do primeiro commit ao Pull Request

Reorganizado por tema, sem repetições, com correções e exemplos práticos em cada comando — baseado nas anotações do curso gratuito de Git e GitHub do Tiago Matos (YouTube).

14 partes | 80+ comandos | exemplo prático em cada item



Sumário

Guia dividido em 14 partes, do terminal ao fluxo completo de Pull Requests no GitHub. Cliques nos itens abaixo levam direto à seção correspondente. Blocos amarelos marcam correções em relação às anotações originais; blocos azuis marcam dicas complementares; cada comando traz um exemplo prático de uso.

01 **Comandos Básicos de Terminal**
Navegação, arquivos e produtividade

02 **Git: Primeiros Passos**
Configuração e inicialização

03 **Git: Staging e Commits**
Do arquivo modificado ao histórico

04 **Git: Branches**
Ramificações e trabalho paralelo

05 **Git: Stash**
Guardando trabalho temporariamente

06 **Git: Desfazendo Alterações**
Checkout, revert, reset e reflog

07 **Git: Merge e Conflitos**
Estratégias de fusão de branches

08 **Git: .gitignore**
Ignorando arquivos do rastreamento

09 **GitHub: Repositórios Remotos**
Push, pull e clone

10 **GitHub: Autenticação SSH**
Configuração segura por máquina

11 **GitHub: Criando um Repositório**
README e templates prontos

12 **GitHub: Fork**
Contribuindo com projetos de terceiros

13 **GitHub: Pull Requests**
Fluxo completo pela interface web

14 **Boas Práticas e Dicas Extras**
Convenções e produtividade

01 Comandos Básicos de Terminal

Navegação, arquivos e produtividade no terminal

1.1 Navegação entre pastas

`pwd`

Exibe o caminho completo (path) da pasta em que você está no momento.

EXEMPLO PRÁTICO

```
$ pwd
/home/usuario/projetos/git-github
```

`ls`

Lista os arquivos e pastas do diretório atual.

EXEMPLO PRÁTICO

```
$ ls
README.md   main.py   src/
```

`ls -a`

Lista todos os arquivos, incluindo os ocultos (como a pasta **.git**).

EXEMPLO PRÁTICO

```
$ ls -a
.   ..   .git   .gitignore   README.md
```

`ls -l`

Lista os arquivos em formato detalhado, com permissões, tamanho e data de modificação.

EXEMPLO PRÁTICO

```
$ ls -l
-rw-r--r-- 1 usuario staff 214 Jul 2 10:15 README.md
```

`cd [pasta]`

Acessa um diretório específico.

EXEMPLO PRÁTICO

```
$ cd projetos/git-github
```

`cd ..`

Sobe um nível na hierarquia de pastas.

EXEMPLO PRÁTICO

```
$ cd ..
```

`cd ~`

Vai direto para a pasta pessoal (home) do usuário, de qualquer lugar do sistema.

EXEMPLO PRÁTICO

```
$ cd ~
```

`cd -`

Volta para o último diretório em que você estava.

EXEMPLO PRÁTICO

```
$ cd -
/home/usuario/projetos/git-github
```

1.2 Criando, movendo e apagando arquivos e pastas

`mkdir [pasta]`

Cria um novo diretório.

EXEMPLO PRÁTICO

```
$ mkdir componentes
```

`touch [arquivo]`

Cria um novo arquivo vazio (ou atualiza a data de modificação de um arquivo existente).

EXEMPLO PRÁTICO

```
$ touch index.html
```

`mv [origem] [destino]`

Move arquivos ou pastas. Também é usado para renomear, movendo para o mesmo local com um novo nome.

EXEMPLO PRÁTICO

```
$ mv relatorio.txt docs/relatorio-final.txt
# move E renomeia ao mesmo tempo
```

`cp [origem] [destino]`

Copia arquivos ou pastas. Use **cp -r** para copiar uma pasta inteira, incluindo o conteúdo dela.

EXEMPLO PRÁTICO

```
$ cp -r src/ backup-src/
```

`rm [arquivo]`

Remove um arquivo específico.

EXEMPLO PRÁTICO

```
$ rm temp.txt
```

`rm -rf [pasta]`

CUIDADO

Remove uma pasta e todo o seu conteúdo, sem pedir confirmação.

EXEMPLO PRÁTICO

```
$ rm -rf build/
```

⚠ Observação: não existe lixeira — a remoção com rm -rf é definitiva. Use com cuidado, principalmente combinado com -f.

`rmdir [pasta]`

Remove uma pasta, mas somente se ela estiver vazia — uma alternativa mais segura ao **rm -rf** quando aplicável.

EXEMPLO PRÁTICO

```
$ rmdir pasta-vazia
```

1.3 Visualizando e editando arquivos

`cat [arquivo]`

Exibe todo o conteúdo de um arquivo diretamente na tela.

EXEMPLO PRÁTICO

```
$ cat README.md
# Meu Projeto
Descrição do projeto...
```

`less [arquivo]`

Abre o conteúdo de um arquivo longo em modo de leitura paginada — útil quando **cat** deixaria a saída ilegível. Use as setas para navegar e **q** para sair (o mesmo vale para outras saídas longas do Git, como **git log** sem **--oneline**).

EXEMPLO PRÁTICO

```
$ less historico.log
```

`nano [arquivo]`

Abre o editor de texto integrado ao terminal. **Ctrl+O** salva (Write Out), **Ctrl+X** sai, **Ctrl+W** busca um termo no arquivo.

EXEMPLO PRÁTICO

```
$ nano .gitignore
# edite o arquivo, depois Ctrl+O, Enter, Ctrl+X
```

`clear`

Limpa a tela do terminal. Atalho equivalente: **Cmd+K** no Mac, ou **Ctrl+L** (funciona também no Windows e no Linux).

EXEMPLO PRÁTICO

```
$ clear
```

1.4 Atalhos e produtividade

`Tab`

Autocompleta comandos, arquivos e pastas. Pressione duas vezes para ver todas as opções quando houver ambiguidade.

EXEMPLO PRÁTICO

```
$ git chec
# pressione Tab...
git checkout
```

`↑ (seta para cima)`

Recupera o último comando digitado; pressione várias vezes para navegar pelo histórico de comandos já executados.

EXEMPLO PRÁTICO

```
# depois de rodar git status, pressione ↑
git status  →  aparece de novo, pronto para reenviar
```

`Ctrl+C`

Interrompe (cancela) um comando em execução no terminal.

EXEMPLO PRÁTICO

```
$ python -m http.server
# servidor rodando... pressione Ctrl+C para parar
```

`cmd1 && cmd2`

Executa o segundo comando somente se o primeiro terminar com sucesso — encadeamento muito usado no dia a dia com Git.

EXEMPLO PRÁTICO

```
$ git add . && git commit -m "ajusta layout do header"
```

02

Git: Primeiros Passos

Verificação, configuração e criação de um repositório

2.1 Verificação e configuração inicial

`git --version`

Verifica se o Git está instalado e exibe a versão atual.

EXEMPLO PRÁTICO

```
$ git --version
git version 2.43.0
```

`git config --global user.name "Nome"`

Define o nome de usuário que aparecerá nos seus commits.

EXEMPLO PRÁTICO

```
$ git config --global user.name "Seu Nome"
```

`git config --global user.email "email"`

Define o e-mail associado aos seus commits (recomenda-se usar o mesmo e-mail da conta do GitHub).

EXEMPLO PRÁTICO

```
$ git config --global user.email "seu_email@exemplo.com"
```

`git config --global --list`

Exibe todas as configurações globais do Git.

EXEMPLO PRÁTICO

```
$ git config --global --list
user.name=Seu Nome
user.email=seu_email@exemplo.com
```

⚠ Observação: nas anotações originais estava `git config --global list`, sem o segundo hífen antes de `list`. A sintaxe correta usa `--list`.

2.2 Iniciando um repositório

`git init`

Inicializa um novo repositório Git no diretório atual, criando a pasta oculta **.git** que armazena todo o histórico do projeto.

EXEMPLO PRÁTICO

```
$ git init
Initialized empty Git repository in /projetos/meu-app/.git/
```

03

Git: Staging e Commits

O ciclo de vida das alterações, do arquivo ao histórico

3.1 Verificando o estado do repositório

`git status`

Mostra o estado atual do repositório: arquivos modificados, arquivos prontos para commit (staged) e arquivos ainda não rastreados.

EXEMPLO PRÁTICO

```
$ git status
On branch main
Changes not staged for commit:
  modified:   main.py
```

`git diff`

Mostra, linha a linha, as diferenças entre os arquivos modificados no diretório de trabalho e a versão em staging (ou do último commit, se nada estiver staged).

EXEMPLO PRÁTICO

```
$ git diff
- print('versão antiga')
+ print('versão nova')
```

`git diff --staged`

Mostra as diferenças entre o que está em staging e o último commit — exatamente o que vai entrar no próximo commit. Também pode ser escrito **git diff --cached**.

EXEMPLO PRÁTICO

```
$ git diff --staged
```

3.2 Adicionando alterações ao staging

`git add [arquivo]`

Adiciona um arquivo específico à área de staging.

EXEMPLO PRÁTICO

```
$ git add main.py
```

`git add .`

Adiciona todos os arquivos modificados ou novos do diretório atual de uma só vez.

EXEMPLO PRÁTICO

```
$ git add .
```

`git rm --cached [arquivo]`

Remove um arquivo do staging (ou para de rastreá-lo, se já estava commitado), mantendo o arquivo intacto no computador. Muito usado quando um arquivo sensível (como **.env**) foi commitado por engano.

EXEMPLO PRÁTICO

```
$ git rm --cached .env
# depois, adicione .env ao .gitignore (Parte 8)
```

3.3 Registrando commits

`git commit -m "mensagem"`

Cria um commit com as alterações que estão no staging, junto de uma mensagem descritiva.

EXEMPLO PRÁTICO

```
$ git commit -m "feat: adiciona tela de login"
```

`git commit --author="Nome <email>"`

Define manualmente a autoria do commit — útil em simulações, correções ou commits feitos em nome de outra pessoa.

EXEMPLO PRÁTICO

```
$ git commit --author="Nome Sobrenome <email@exemplo.com>" -m "fix: corrige bug no header"
```

`git commit`

Sem a flag **-m**, abre o editor de texto padrão do terminal (geralmente o Vim) para escrever a mensagem.

EXEMPLO PRÁTICO

```
$ git commit
# abre o Vim → digite a mensagem → :wq + Enter
```

⚠ Observação: para salvar e sair do Vim, digite :wq e pressione Enter. O atalho :q das anotações originais só funciona se o texto padrão não tiver sido alterado; se você editar a mensagem, o Vim recusa fechar só com :q.

3.4 Histórico de commits

git log

Exibe o histórico completo de commits: hash (ID), autor, data e mensagem.

EXEMPLO PRÁTICO

```
$ git log
commit a1b2c3d...
Author: Seu Nome <email>
Date:   Wed Jul 2 10:00:00 2026
    feat: adiciona tela de login
```

git log --oneline

Exibe o mesmo histórico de forma resumida: início do hash e mensagem, um commit por linha.

EXEMPLO PRÁTICO

```
$ git log --oneline
a1b2c3d Corrige bug no formulário de login
e4f5g6h Adiciona validação de e-mail
i7j8k9l Primeiro commit
```

04

Git: Branches (Ramificações)

Linhas independentes de desenvolvimento

Uma branch é uma linha independente de desenvolvimento. Alterações feitas em uma branch específica não afetam a branch principal (**main** ou **master**) até que aconteça um merge.

`git branch`

Lista todas as branches do repositório, marcando a atual com um asterisco (*).

EXEMPLO PRÁTICO

```
$ git branch
  feature/login
* main
```

`git branch [nome]`

Cria uma nova branch a partir da branch atual, sem mudar para ela.

EXEMPLO PRÁTICO

```
$ git branch feature/login
```

`git checkout [branch]`

Alterna para a branch informada.

EXEMPLO PRÁTICO

```
$ git checkout feature/login
```

`git checkout -b [nome]`

Cria uma nova branch e já muda para ela — atalho para **git branch** + **git checkout**.

EXEMPLO PRÁTICO

```
$ git checkout -b feature/dark-mode
```

`git branch -d [nome]`

SEGURO

Deleta uma branch de forma segura, bloqueando a ação se houver commits ainda não mesclados (merged).

EXEMPLO PRÁTICO

```
$ git branch -d feature/login
```

`git branch -D [nome]`

CUIDADO

Força a exclusão da branch, descartando qualquer alteração não mesclada. Use com cautela.

EXEMPLO PRÁTICO

```
$ git branch -D experimento-quebrado
```

💡 Dica: Desde a versão 2.23, o Git introduziu `git switch` e `git restore` para dividir as funções acumuladas pelo `checkout`. `git switch [branch]` equivale a `git checkout [branch]`, e `git switch -c [nome]` equivale a `git checkout -b [nome]`. Nenhum dos dois é obrigatório — o checkout tradicional continua funcionando normalmente.

EXEMPLO — ALTERNATIVA MODERNA

```
$ git switch main
$ git switch -c hotfix/navbar
```

05

Git: Guardando Trabalho Temporariamente

O comando stash

Às vezes é preciso trocar de branch sem ainda ter terminado (nem querer commitar) o que está sendo feito. O **git stash** guarda essas alterações de lado temporariamente.

`git stash`

Guarda todas as alterações não commitadas (staged ou não) e limpa o diretório de trabalho, deixando-o como no último commit.

EXEMPLO PRÁTICO

```
$ git stash
Saved working directory and index state WIP on main: a1b2c3d ajusta header
```

`git stash pop`

Restaura as alterações mais recentes guardadas no stash e as remove da lista.

EXEMPLO PRÁTICO

```
$ git stash pop
```

`git stash list`

Lista todos os stashes guardados, caso haja mais de um.

EXEMPLO PRÁTICO

```
$ git stash list
stash@{0}: WIP on main: a1b2c3d ajusta header
```

06

Git: Desfazendo Alterações

Três níveis de risco, e uma rede de segurança

6.1 Três formas de desfazer, por nível de risco

`git checkout [commit]` — TOTALMENTE SEGURO

Permite "viajar no tempo" para ver como o código estava em um commit específico, sem alterar o histórico. Para voltar ao presente, use **git checkout [nome_da_branch]**.

EXEMPLO PRÁTICO

```
$ git checkout a1b2c3d
# ...explore o código...
$ git checkout main
```

`git revert [commit]` — MODERADAMENTE SEGURO

Cria um novo commit que anula as alterações de um commit anterior, mantendo todo o histórico intacto. Ideal para desfazer algo em um repositório compartilhado.

EXEMPLO PRÁTICO

```
$ git revert a1b2c3d
```

`git reset --hard [commit]` — PERIGOSO

Move o ponteiro da branch para um commit anterior, apagando os commits posteriores e todas as alterações do diretório de trabalho.

EXEMPLO PRÁTICO

```
$ git reset --hard a1b2c3d
```

6.2 Entendendo o git reset a fundo

O git reset tem três variações, que diferem no que acontece com as alterações depois de mover o ponteiro da branch:

`git reset --soft [commit]`

Move o ponteiro, mas mantém todas as alterações no staging — como se você tivesse acabado de dar **git add** em tudo de novo.

EXEMPLO PRÁTICO

```
$ git reset --soft HEAD~1
# desfaz o último commit, mantém tudo staged
```

`git reset --mixed [commit]`

Comportamento padrão do **git reset** (sem flag). Move o ponteiro e tira as alterações do staging, mas mantém tudo no diretório de trabalho como alterações não commitadas.

EXEMPLO PRÁTICO

```
$ git reset HEAD~1
# equivalente a --mixed
```

`git reset --hard [commit]`

PERIGOSO

Move o ponteiro e descarta completamente as alterações, tanto do staging quanto do diretório de trabalho.

EXEMPLO PRÁTICO

```
$ git reset --hard HEAD~1
```

6.3 Rede de segurança: git reflog

`git reflog`

Mostra um histórico de tudo por onde o HEAD do repositório passou — incluindo commits "perdidos" após um **reset --hard** ou uma branch deletada. Esses commits costumam continuar recuperáveis por semanas antes de serem descartados de vez.

EXEMPLO PRÁTICO

```
$ git reflog
e4f5g6h HEAD@{1}: commit: feat: adiciona checkout
a1b2c3d HEAD@{0}: reset: moving to HEAD~3
$ git reset --hard e4f5g6h
# recupera o commit "perdido"
```

07

Git: Merge e Resolução de Conflitos

Três estratégias para unir branches

7.1 Estratégias de merge

1 Merge Commit (padrão)

Preserva todo o histórico original exatamente como aconteceu.

Como funciona: o Git cria um commit especial (com dois "pais") que une os históricos das duas branches.

Vantagens: mantém rastreabilidade exata de quem fez cada commit e quando.

Desvantagens: em projetos com muitos merges, o histórico pode ficar poluído e difícil de ler.

EXEMPLO PRÁTICO

```
$ git checkout main
$ git merge feature/login
# cria um commit de merge automaticamente
```

2 Squash and Merge

Condensa todos os commits da branch em um único commit antes de unir as branches.

Como funciona: o Git junta todas as alterações da branch em um só "commit resumido" e o adiciona na branch de destino.

Vantagens: mantém o histórico da branch principal limpo e linear; permite usar commits de rascunho ("wip", "ajuste") na branch de desenvolvimento sem que apareçam no histórico oficial.

Desvantagens: perde-se o histórico detalhado dos passos intermediários.

EXEMPLO PRÁTICO

```
$ git checkout main
$ git merge --squash feature/login
$ git commit -m "feat: sistema de login completo"
```

3 Rebase

Reescreve o histórico, colocando seus commits no topo da branch de destino.

Como funciona: o Git "descola" os seus commits, atualiza a branch com os commits mais recentes da main, e reaplica os seus commits um por um, como se tivessem sido feitos agora.

Vantagens: histórico perfeitamente linear, sem commits de merge extras; facilita a leitura e o uso do **git bisect** (Parte 14).

Desvantagens: reescreve o histórico (os hashes mudam), o que exige **push --force** e pode sobrescrever o trabalho de outras pessoas. Prefira **--force-with-lease**: ele verifica se ninguém mais alterou a branch remota antes de sobrescrever.

EXEMPLO PRÁTICO

```
$ git checkout feature/login
$ git rebase main
# resolva conflitos se aparecerem, depois:
$ git push --force-with-lease
```

7.2 Fast-forward vs. Recursive

Essas são as estratégias internas que o Git escolhe automaticamente ao rodar **git merge**. No **fast-forward**, a branch de destino simplesmente "avança" até o último commit da outra branch — só é possível quando não houve novos commits na branch de destino nesse meio-tempo. Na **recursive**, o Git precisa de fato combinar históricos divergentes, criando um commit de merge. O resultado final (a união do código) é o mesmo; a diferença está em como o histórico fica registrado.

7.3 Resolvendo conflitos

`git merge [branch]`

Tenta fundir a branch informada na branch atual. Se houver alterações conflitantes na mesma linha de código, o Git pausa o processo e pede resolução manual.

EXEMPLO PRÁTICO

```
$ git merge feature/login
CONFLICT (content): Merge conflict in header.html
```

`git add .`

Usado após corrigir os conflitos (manualmente ou pela IDE), marca os arquivos como resolvidos.

EXEMPLO PRÁTICO

```
$ git add .
```

`git commit`

Finaliza o merge. Sem a flag **-m**, abre o editor com uma mensagem já sugerida — nesse caso, é boa prática aceitar o texto padrão, pois ele documenta a fusão de branches.

EXEMPLO PRÁTICO

```
$ git commit
# aceita a mensagem padrão: Merge branch feature/login
```

No VS Code (e IDEs semelhantes)

Accept Current Change	Mantém o código da branch em que você está.
Accept Incoming Change	Aceita o código da branch que está sendo mesclada.
Accept Both Changes	Mantém os dois blocos de código.
Apply	Aplica a resolução escolhida depois de editar manualmente as linhas.

08

Git: Ignorando Arquivos

O arquivo .gitignore

`touch .gitignore`

Cria o arquivo oculto **.gitignore** na raiz do projeto.

EXEMPLO PRÁTICO

```
$ touch .gitignore
```

Sintaxe: cada linha define o que ignorar

senha.txt	Ignora apenas aquele arquivo específico.
node_modules/	Ignora a pasta inteira (a barra no final indica que é um diretório).
***.log**	Ignora todos os arquivos com aquela extensão, em qualquer pasta do projeto.

EXEMPLO PRÁTICO — .GITIGNORE COMPLETO

```
# Exemplo de .gitignore para um projeto real
node_modules/
__pycache__/
.env
dist/
build/
*.log
.DS_Store
.vscode/
```

⚠ Observação: o .gitignore só funciona para arquivos que o Git ainda não está rastreando. Se um arquivo já foi commitado antes de entrar no .gitignore, adicioná-lo à lista não é suficiente — é preciso rodar git rm --cached [arquivo] (Parte 3.2) para que o Git realmente pare de monitorá-lo.

09

GitHub: Repositórios Remotos

Conectando, enviando e recebendo alterações

9.1 Conectando um repositório remoto

`git remote add origin [url]`

Cria um "apelido" (**origin**, por convenção da comunidade) para o endereço do repositório remoto.

EXEMPLO PRÁTICO

```
$ git remote add origin git@github.com:usuario/repositorio.git
```

`git remote -v`

Lista os repositórios remotos configurados, mostrando os endereços de fetch e push.

EXEMPLO PRÁTICO

```
$ git remote -v
origin  git@github.com:usuario/repositorio.git (fetch)
origin  git@github.com:usuario/repositorio.git (push)
```

9.2 Enviando e recebendo alterações

`git push`

Envia os commits locais para o servidor remoto.

EXEMPLO PRÁTICO

```
$ git push
```

`git push origin main`

Envia os commits para a branch main do remoto chamado origin.

EXEMPLO PRÁTICO

```
$ git push origin main
```

`git push -u origin main`

Igual ao anterior, mas grava **main** como a branch padrão associada (upstream). Depois disso, um simples **git push** já sabe para onde enviar.

EXEMPLO PRÁTICO

```
$ git push -u origin main
# da próxima vez, basta:
$ git push
```

`git pull`

Baixa e integra as alterações mais recentes do repositório remoto na cópia local. É a junção de **git fetch** + **git merge**.

EXEMPLO PRÁTICO

```
$ git pull
```

`git pull origin main`

Mesma coisa, especificando explicitamente o remoto e a branch.

EXEMPLO PRÁTICO

```
$ git pull origin main
```

9.3 Clonando repositórios

`git clone [url]`

Baixa uma cópia completa de um repositório do GitHub para a máquina local.

EXEMPLO PRÁTICO

```
$ git clone git@github.com:usuario/repositorio.git
```

`git clone [url] [pasta]`

Clona o repositório definindo um nome personalizado para a pasta local.

EXEMPLO PRÁTICO

```
$ git clone git@github.com:usuario/repositorio.git meu-app
```

10

GitHub: Autenticação SSH

Configuração única por máquina

Configurar uma chave SSH evita ter que digitar usuário e senha (ou token) toda vez que você faz push ou pull. É uma configuração feita uma única vez por máquina.

1

Gerar o par de chaves

ed25519 é o algoritmo recomendado atualmente pelo GitHub. Aceite o local padrão com Enter; a senha (passphrase) é opcional.

```
$ ssh-keygen -t ed25519 -C "seu_email@exemplo.com"
```

2

Iniciar o ssh-agent

Inicia o programa que gerencia suas chaves na sessão atual.

```
$ eval "$(ssh-agent -s)"
```

3

Adicionar a chave ao agent

Adiciona sua chave privada ao ssh-agent.

```
$ ssh-add ~/.ssh/id_ed25519
```

4

Copiar a chave pública

Exibe a chave pública no terminal, para copiar.

```
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5... seu_email@exemplo.com
```

5

Cadastrar no GitHub

GitHub → Settings → SSH and GPG keys → New SSH key. Cole a chave pública copiada no passo anterior.

6

Testar a conexão

Se aparecer uma mensagem de boas-vindas com seu usuário, a configuração deu certo.

```
$ ssh -T git@github.com
Hi usuario! You've successfully authenticated...
```

11 GitHub: Criando um Repositório

README e templates prontos

README.md

Arquivo de documentação que aparece na página inicial do repositório no GitHub — o primeiro contato de quem visita o projeto. Vale a pena descrever bem o que o projeto faz, como instalar e como usar.

EXEMPLO PRÁTICO — ESTRUTURA MÍNIMA

```
# Nome do Projeto
## O que é
Breve descrição do projeto.
## Como instalar
git clone ...
```

.gitignore (template)

Ao criar um repositório pelo site do GitHub, é possível escolher um template pronto de **.gitignore** de acordo com a linguagem do projeto (Node, Python, Java, etc.), já configurado com os padrões mais comuns a ignorar.

EXEMPLO PRÁTICO

```
# na tela de criação: "Add .gitignore" → selecione "Python", "Node" ou a linguagem do seu projeto
```

12

GitHub: Fork

Contribuindo com projetos de terceiros

Fork e clone são frequentemente confundidos:

Clone	Baixa uma cópia de um repositório para a sua máquina. Sem permissão de escrita no repositório original, não é possível dar push nele.
Fork	Cria uma cópia completa do repositório na sua própria conta do GitHub. A partir daí, você clona o seu fork (não o original), faz alterações livremente, e usa um Pull Request para propor que suas mudanças sejam incorporadas ao repositório original.

Fluxo típico de contribuição (open source)

EXEMPLO PRÁTICO

```
# 1. Fork pelo botão no GitHub → usuario/repo vira seu-usuario/repo
$ git clone git@github.com:seu-usuario/repositorio.git
$ cd repositorio
$ git checkout -b feature/melhoria
# ... alterações e commits ...
$ git push origin feature/melhoria
# depois, abra o Pull Request pelo GitHub, do seu fork para o repositório original
```

13

GitHub: Pull Requests

Fluxo completo pela interface web

1

Criação do PR

Depois do push, o GitHub exibe o botão "Compare & pull request", que abre a solicitação de fusão.

2

Revisão e feedback

Quem mantém o projeto pode analisar os commits e deixar comentários, visíveis diretamente na interface do Pull Request.

3

Atualização automática

Se novos commits forem feitos na mesma branch (novo add, commit e push), o PR já aberto é atualizado automaticamente.

4

Merge

Após a aprovação, o botão "Merge pull request" finaliza a integração. Em seguida, dá para deletar a branch original pelo próprio GitHub.

EXEMPLO PRÁTICO — DO PUSH AO MERGE

```
$ git push origin feature/dark-mode
# no GitHub: Compare & pull request → descreva as mudanças → Create pull request
# após aprovação: Merge pull request → Delete branch
```

14

Boas Práticas e Dicas Extras

Para além do básico

Trabalhe em branches próprias

Evite commitar direto na main/master, mesmo trabalhando sozinho. Crie uma branch por funcionalidade ou correção — isso facilita a revisão, reverte erros com mais segurança e é essencial ao trabalhar em equipe.

EXEMPLO PRÁTICO

```
$ git checkout -b feature/nova-funcionalidade
```

Conventional Commits

Um padrão bastante adotado no mercado para mensagens de commit, que deixa o histórico mais legível e é frequentemente esperado em vagas e projetos profissionais:

feat	nova funcionalidade
fix	correção de bug
docs	alterações apenas na documentação
style	formatação, sem mudança de lógica
refactor	reorganização do código sem alterar comportamento
test	criação ou ajuste de testes
chore	tarefas de manutenção (dependências, configuração, etc.)

EXEMPLO PRÁTICO

```
$ git commit -m "feat: adiciona autenticação de usuário com JWT"
```

git bisect

Faz uma busca binária pelo histórico de commits para encontrar qual commit introduziu um bug. Um dos motivos para preferir um histórico linear (Parte 7, Rebase).

EXEMPLO PRÁTICO

```
$ git bisect start
$ git bisect bad
# commit atual está com bug
$ git bisect good v1.2.0
# essa versão funcionava — o Git testa os commits do meio
$ git bisect good
# (ou bad) até encontrar o culpado
$ git bisect reset
# volta ao estado original
```

Encadeamento (&&)

Já visto na Parte 1 — vale reforçar como hábito de produtividade.

EXEMPLO PRÁTICO

```
$ git add . && git commit -m "mensagem" && git push
```